Home Sensor w/ Reasoning

Team EcoNest

Problem Background

Context

- Modern homes increasingly rely on multiple smart devices that provide basic notifications but lack the ability to perform **contextual reasoning across devices.**

Why it matters

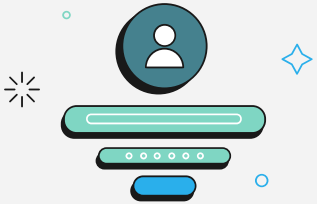
- Homeowners lack clear insight into their home's **energy consumption and security status**, making it difficult to:
  - Identify inefficiencies
  - Detect meaningful events
  - Make informed decisions

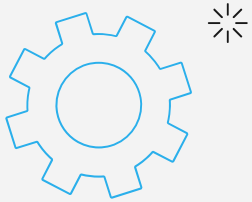
Current Limitations

- Smart device platforms typically provide **raw, unprocessed sensor data**
- Limited ability to **correlate data across devices**
- Lack of **prioritization or intelligent interpretation of events**

Key statistics

- Idle energy consumption accounts for **5–10% of residential energy usage**.
- A burglary occurs roughly every **25–30 seconds in the U.S.**
- The smart home market is expected to grow at a **27.07% CAGR over the next 5 years**, increasing the need for **integrated, intelligent monitoring solutions.**





Needs Statement

Who has the need?

- Homeowners using multiple smart home devices (e.g., smart plugs, motion sensors, sound sensors) who require **meaningful insights and reasoning** about their home's energy usage and security.

What is missing?

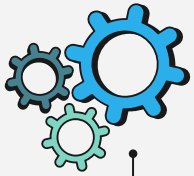
- A **centralized system** that:
  - Aggregates data from diverse smart devices.
  - Interprets sensor data across sources.
  - Provides **contextual reasoning** for energy consumption and security events.

Why is a solution necessary?

- Despite widespread adoption of smart devices:
  - Data is **fragmented across multiple platforms.**
  - Users receive **raw sensor output without context or prioritization.**
  - Homeowners lack the ability to **assess risks or inefficiencies effectively.**

Impact if unmet

- Delayed or missed responses to **security threats.**
- Increased **energy inefficiency and costs.**
- Continued reliance on **disconnected data with no actionable guidance.**



Goals and Objectives

Main Goal:

- Develop a **centralized smart home monitoring system** that applies data driven reasoning to optimize energy consumption and enhance home security.

Objectives:

- Collect and integrate **multi-modal sensor data** (energy, motion, sound) for analysis and model training.
- Enable **reliable and secure data transmission** from devices to the central system.
- Detect anomalies in real time to identify:
  - Abnormal energy usage
  - Potential security threats
- Analyze historical data to identify trends and support **predictive decision-making**.
- Generate actionable **recommendations** for improving energy efficiency and security.
- Provide users with a **visual interface** to monitor system status and receive timely notifications.

Energy Requirements

Energy Collection

- The system shall collects energy data from each monitored device **every 30 seconds**
- Each device shall report:
  - **Voltage (V)**
  - **Current (A)**
  - **Power (W)**
  - **Total Energy Consumption (kWh)**

Device Monitoring

- The system shall support monitoring of **multiple devices** over WiFi
- Each device shall have a **unique identifier** associated with a specific home/unit

Data Storage

- The system shall transmit all sensor data to a **central SQL database.**
- Each data entry shall include:
  - Timestamp (universal time)
  - Device Identifier
- The system shall maintain a **continuous historical record** of energy usage.

Timestamp	Device	Voltage	Current	Power	Total Energy
2026-03-04 15:32:00	Washer	121 V	3.20 A	387 W	0.56 kWh

Security Requirements

Environmental Monitoring

- Motion and sound sensors shall operate continuously (24/7) while powered.

Sound Detection

- The system shall measure environmental noise in **decibels (dB)**
- The system shall log events when noise exceeds a configurable threshold (30 dB)

Motion Detection

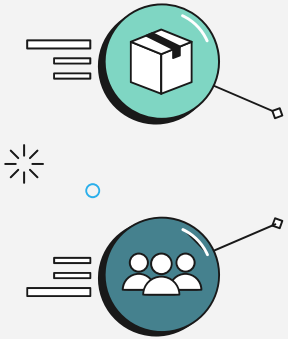
- The system shall detect motion within sensor range.
- The system shall log:
  - Time of detection
  - Duration of motion

Event Logging

- The system shall timestamp all motion and sound events.
- The system shall correlate motion and sound data to identify potential activity patterns.

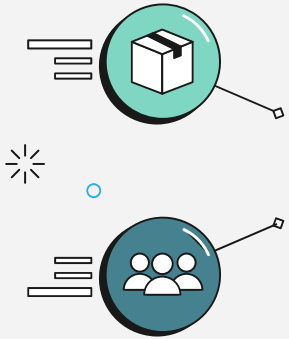
AI/ML Integration

- The system shall provide collected sensor and event data as input to AI/ML model.
- The AI/ML model shall perform reasoning such as anomaly detection or activity inference.

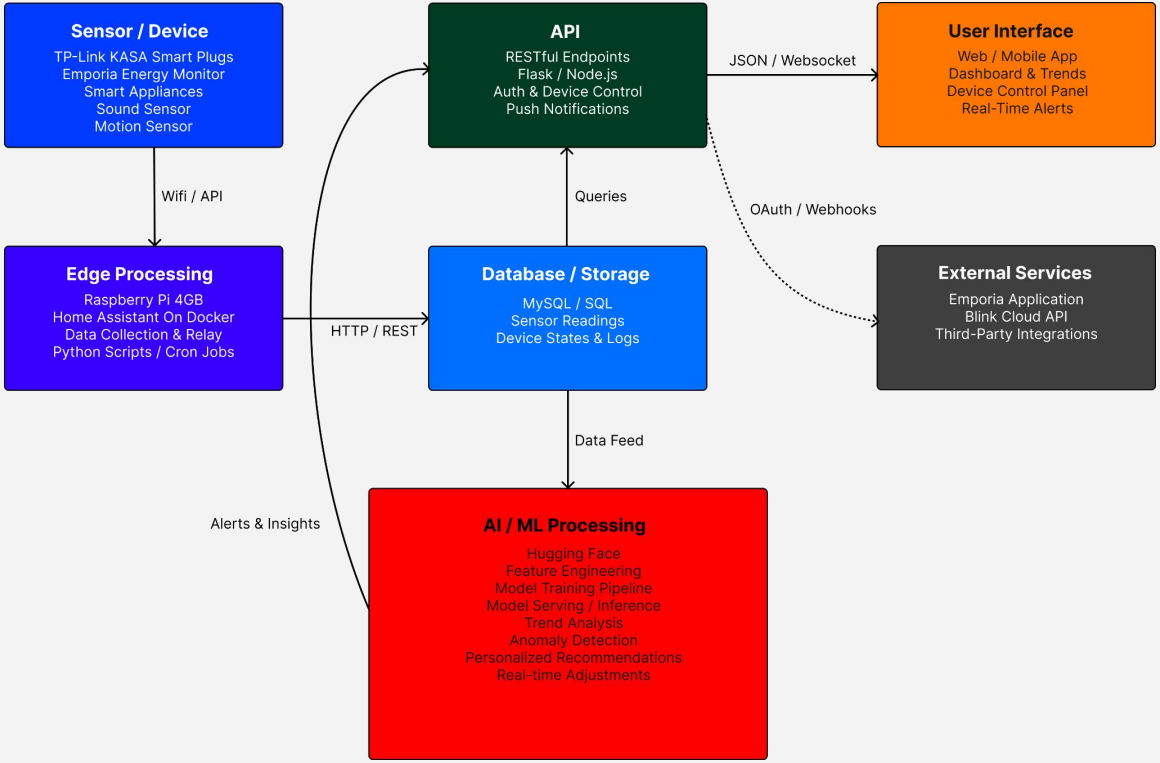


Design Alternatives

- **Cloud Database (AWS)**: Moved from **local storage** to **AWS** for more reliable data storage and easier access for analysis
- **Home Assistant via Docker on Raspberry Pi**: Replaced a rigid deployment setup with **Docker** for a more flexible environment and **remote** device **access**
- **Breaker-Level Energy Monitoring**: Switched from **smart** plugs to **measurement** tools connected directly to electrical breakers for more **accurate** and **comprehensive** energy tracking
- **Sound Microphones over Sound Sensors**: Original **sound sensing** approach could not produce meaningful data, so we switched to **microphones** plugged **directly** into the **Raspberry Pi** to capture sound **magnitude** effectively
- **Motion Sensor Hub Requirement**: Discovered that **motion sensors** for Units 2 and 3 require a **hub** to operate; resolved by leveraging **Home Assistant** compatibility to **integrate** them into the existing **architecture**



System-Level Description



Sensors/Devices

- **Kasa Smart Plugs:**
  - Captures devices energy output
  - (Current, Voltage, Power, etc.)
  - Currently set up in unit 1 or 2
- **Emporia Vue Energy Monitor + Emporia Energy Plugs**
  - Installed directly into the breaker box
  - Clamps onto each individual breaker and measures energy output (voltage, current, power, energy usage (kWh))
  - Gives a per-appliance data
  - Current set up in unit 3
- **Zigbee Dongle and Presence Detector**
  - Detects motion using infrared (body heat)
  - Gives data of motion detected (1) or no motion detected (0)
- **Microsoft USB Microphone**
  - Detects sound based on environment and tracks if there is a spike is noise, which will be logged

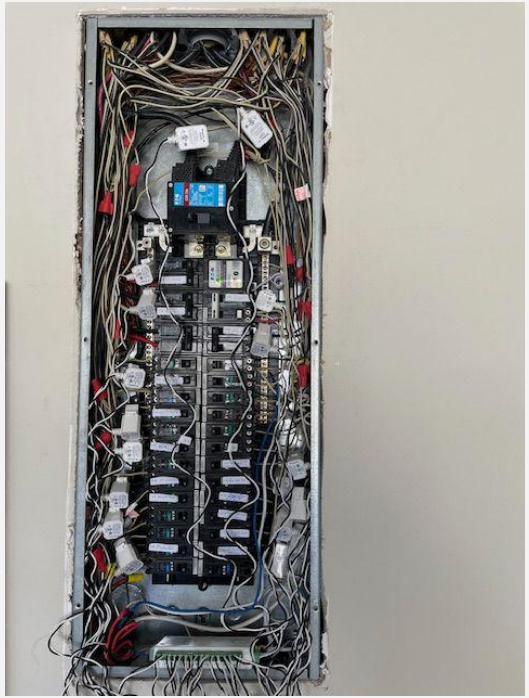
Sensor / Device

TP-Link KASA Smart Plugs
Emporia Energy Monitor
Smart Appliances
Sound Sensor
Motion Sensor

All of these are either physically connected or managed off the Raspberry Pi

Pictures

Energy Monitoring



Security (Motion + Sound)

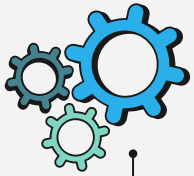
+

Testing/Validation

- **Smart Plug Energy Monitoring**
  - Test:
    - Connect appliances with known power consumption.
    - Compare smart plug readings with expected wattage.
  - Validation:
    - Energy readings are within +/- 10% of expected values.
    - Continuous monitoring works without data loss.
- **Motion Sensor Detection**
  - Test:
    - Perform controlled movements in area at different locations and angles.
  - Validation:
    - Motion is detected.
    - Detection accuracy greater than 90%
- **Sound Sensor Detection**
  - Test:
    - Simulate normal and abnormal sound levels (loud noise, glass breaking, etc.)
  - Validation:
    - Sensor differentiates between normal and abnormal sound levels.

Sensor / Device

TP-Link KASA Smart Plugs
Emporia Energy Monitor
Smart Appliances
Sound Sensor
Motion Sensor



Edge Processing

Role in Our System

- Local decision-making for real-time security events
- Reduces latency (no cloud delay for alerts/actions)
- Maintains functionality even if internet is down

What Runs on the Edge

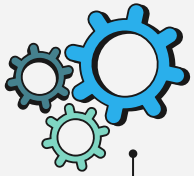
- **Home Assistant (Dockerized)**
  - Device coordination and automation engine
- **Sensor Data Collection & Filtering**
  - Motion, sound, camera triggers processed locally
- **Event Detection Logic (Python / Cron Jobs)**
  - Detect suspicious patterns (unexpected motion, noise spikes)
- **Data Relay**
  - Sends relevant events to cloud/dashboard (not raw data)

Edge Processing

Raspberry Pi 4GB
Home Assistant On Docker
Data Collection & Relay
Python Scripts / Cron Jobs

Why Edge Processing Matters

- Faster alerts (instant response vs cloud round-trip)
- Improved privacy (sensitive data stays local)
- Lower bandwidth usage



Testing/Validation

End-to-End System Testing

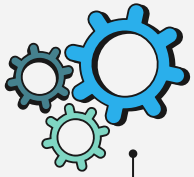
- Simulated real home scenarios (motion, sound, energy)
- Verified full pipeline:
  Sensors → Home Assistant → API → Database

Key Tests Performed

- **Motion Detection:**
  - Controlled movement at different angles and distances
- **Sound Detection:**
  - Normal vs abnormal noise (e.g., loud spikes)
- **Energy Monitoring:**
  - Compared readings against known appliance usage

Edge Processing

Raspberry Pi 4GB
Home Assistant On Docker
Data Collection & Relay
Python Scripts / Cron Jobs



Testing/Validation

System Validation

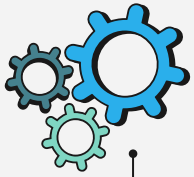
- Accurate detection across all sensor types (>90%)
- Energy readings within ±10% of expected values
- Reliable data transmission with no significant loss
- Alerts triggered correctly for abnormal events

Performance Results

- Near real-time alert response (< 2 seconds)
- Stable operation under continuous data collection

Edge Processing

Raspberry Pi 4GB
Home Assistant On Docker
Data Collection & Relay
Python Scripts / Cron Jobs



Database: Overview

Database / Storage

MySQL / SQL
Sensor Readings
Device States & Logs

Platform Selection:

- The system uses a **MySQL relational database** hosted on **Amazon RDS (AWS)**

Stored Data:

- The database maintains structured records for:
  - **Homes** (user environments)
  - **Devices** (associated with each home)
  - **Sensor Readings** (time-series energy and environmental data)

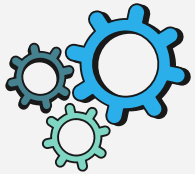
Data Representation:

- The data is stored using **MySQL JSON fields** to support **flexible schemas** for different device types

Design Rationale:

- **Relational Structure** supports clear relationships between homes, devices, and data
- **Efficient Querying** enables analytics, trend analysis, and AI/ML processing
- **Cloud Hosting** provides scalability, reliability, and managed backups

The system is designed to handle high-frequency time-series data.



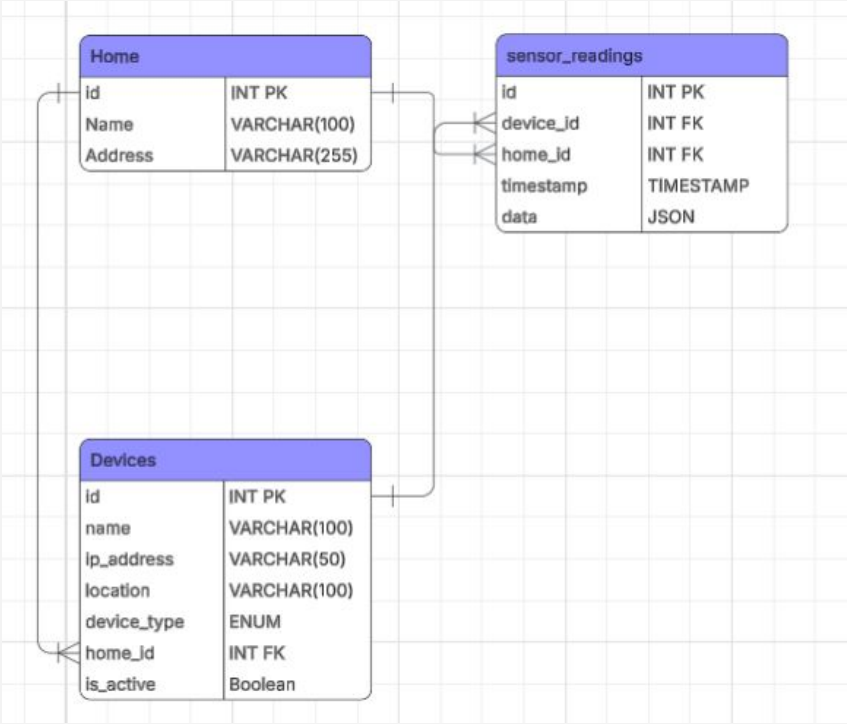
Database: ER Diagram

Database Schema

- Tables:
  - Homes
  - Devices
    - Each device relates to a home
  - Sensor Readings
    - Each sensor reading relates to a device and a home

AI/ML Data Support

- We plan to add a table capable of storing data required for our AI/ML models.
- If future AI workloads require more complex relationships, the system is designed to integrate additional data stores as a graph database without disrupting core architecture



Database: Performance & Scalability

Expected Workload:

- Our focus will be on unit 3 which will contain **30 devices**.
- We are going to **sample every 30 seconds** for the **smart plugs which are 28** of them.
- This results in about **3,360 readings per hour**

Write Performance:

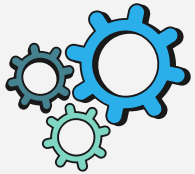
- The system will have **frequent inserts** (every 30 seconds per device)
- It will be **insert only** and not updates to minimize write overhead

Read Performance:

- We use indexing to speed up querying (device id and timestamp, home id and timestamp)

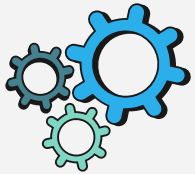
Scalability Strategy:

- Using Amazon RDS supports vertical scalability allowing for upgrading instances and storage size.



Database: Design Tradeoffs

Decision	Alternative	Rationale	Tradeoff
SQL (MySQL)	NoSQL	Structured data, strong relationships	Less flexible
JSON fields	Fully normalized	Flexible schema	Complex queries
AWS RDS	Local DB	Scalable, managed	Cost



Database: Validation Plan

Data Ingestion:

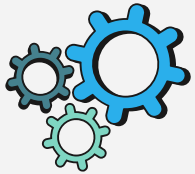
- Test: Verify sensor data is received every 30 seconds
- Method: Compare timestamps of incoming data
- Success Criteria: **≥98% of expected readings received**

Data Storage:

- Test: Ensure all incoming data is stored correctly
- Method: Insert known data and query database
- Success Criteria: **100% data accuracy**, no loss

System Performance:

- Test: Measure database performance under load
- Method: Simulate multiple devices sending data
- Success Criteria:
  - Event detection latency: **< 2 seconds**
  - Data insertion latency: **< 1 second per reading**



API: Overview

Purpose:

- Acts as a **central interface** between system components
- Enables communication between:
  - Sensors/Devices
  - Database
  - UI
  - AI/ML Processing

Architecture:

- Implemented as a **RESTful API**
- Use standard **HTTP** methods (GET, POST)
- **Stateless** design for scalability and simplicity

Core Responsibilities:

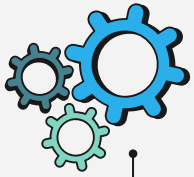
- Receive and validate sensor data from devices
- Store data in database
- Provide data access for UI and AI/ML processing

Design Rationale:

- REST was chosen for:
  - Simplicity and ease of implementation
  - Scalability in distributed systems

API

RESTful Endpoints
Flask / Node.js
Auth & Device Control
Push Notifications



API: Endpoints

Device Management:

- /devices/list
  - GET
  - Lists all devices
- /devices/add
  - POST
  - Adds a device

Sensor Data Inserts:

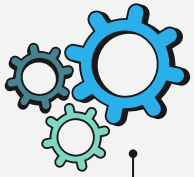
- /readings/add
  - POST
  - Adds all readings given
- /readings/list/<device_id>
  - GET
  - Lists all readings of a given device

Structure:

- All endpoints use JSON for data exchange

API

RESTful Endpoints
Flask / Node.js
Auth & Device Control
Push Notifications



AI/ML Pipeline Overview

Architecture:

- The system uses a multi-stage AI reasoning pipeline hosted on **AWS**, with **Amazon Bedrock** as the LLM integration layer and **MySQL** on **Amazon RDS** as the data backbone

Data Flow:

- The pipeline processes incoming **sensor** readings through **four** stages before reaching the **reasoning** agent:
  - **Sensor Readings** (raw motion, sound, and power data)
  - **Trigger Check** (**rule-based** gate that determines if a **reading** is worth **reasoning** about)
  - **HomeContext JSON** (structured **input** assembled from all data **sources** and sent to the **LLM**)
  - **Reasoning Agent** (Amazon **Bedrock** with **Claude** 3 **Haiku** producing **alerts** and **recommendations**)

AI/ML Pipeline Overview

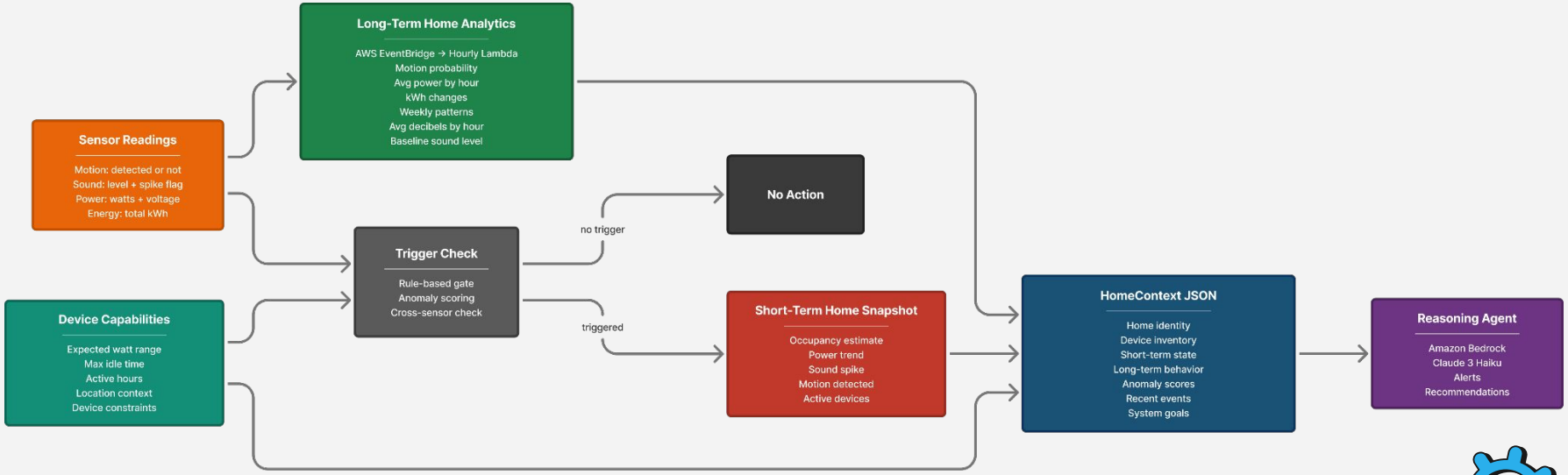
Supporting Data Sources:

- **Short-Term Home Snapshot:** captures current home state including **occupancy** estimate, **power** trend, **sound** spikes, and **active** devices
- **Long-Term Home Analytics:** stores **hourly** precomputed **baselines** via **AWS EventBridge Lambda** including **motion probability**, **avg power**, and **baseline sound** levels
- **Device Profiles:** store static per-device constraints such as expected **watt range**, **max** idle time, and **active hours** for anomaly thresholds

Design Rationale:

- **Trigger-based LLM invocation** ensures the LLM (Claude 3 Haiku) is called only for genuine **anomalies** or scheduled **daily** reviews, keeping costs **low** and responses **fast**
- **Separation of short-term and long-term data** allows the **Reasoning Agent** to distinguish between **current** home **state** and **historical** behavioral **norms**
- **Structured JSON output** ensures **alerts** and **recommendations** are always consistent for the **user interface** and **SMS**

System-Level Description- AI/ML



Validation/Test Plan

Energy Consumption

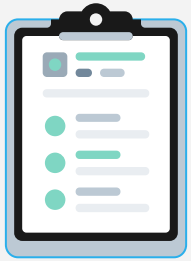
- Simulate a device drawing 4x above its hourly baseline — verify alert is triggered with correct severity
- Simulate a normal HomeContext with no anomalies: verify routine recommendation is returned

Security

- Simulate unexpected motion during overnight hours with occupancy set to unoccupied
- Simulate concurrent sound spike and motion event — verify cross-sensor check escalates correctly
- Dry run flag used to test automation output without implementing it

Final Validation

- ModelI accurately classifies anomaly type based on context
- Model correctly distinguishes between event-driven alerts, routine recommendation, and automation mdoes
- Model correctly incorporates device schedule constraints when classifying on/off-schedule devices
- SMS gate validated independently: suppresses SMS when severity is not HIGH regardless of model output



User Interface

Dashboard

- Build using Home Assistant's built-in Lovelace interface
- Displays real-time energy usage, motion, and sound sensor data across all monitored units
- Customizable cards and views per household unit for easy navigation

Alerts and Notifications

- SMS notifications sent via a third-party SMS service (e.g. Twilio) for detected anomalies
- Alerts triggered by abnormal energy usage, unexpected motion, or sound events
- Configurable alert thresholds allow users to define what constitutes an anomaly

Remote Access & Scalability

- Accessible remotely via Tailscale for secure out-of-home monitoring
  Supports multiple household units from a single Home Assistant interface

Test

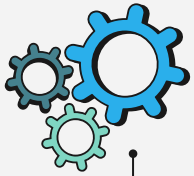
- Verify real-time sensor data updates correctly on the Lovelace dashboard
- Trigger a known anomaly and confirm SMS notification is sent

Validation

- Data refreshes with no stale readings displayed
- SMS received within a reasonable time of the event being detected

User Interface

Web / Mobile App
Dashboard & Trends
Device Control Panel
Real-Time Alerts



External Services

Device Integration

- TP-Link Kasa smart plugs integrate directly with Home Assistant via the native Kasa integration
- Emporia Energy Monitor connects to Home Assistant to capture breaker-level energy consumption for Unit 3

Data Collection

- Home Assistant polls connected devices periodically
- Collects voltage, current, power, and kWh readings

Network & Connectivity

- Tailscale provides secure remote connectivity
- Links household Pi units to the Home Assistant dashboard

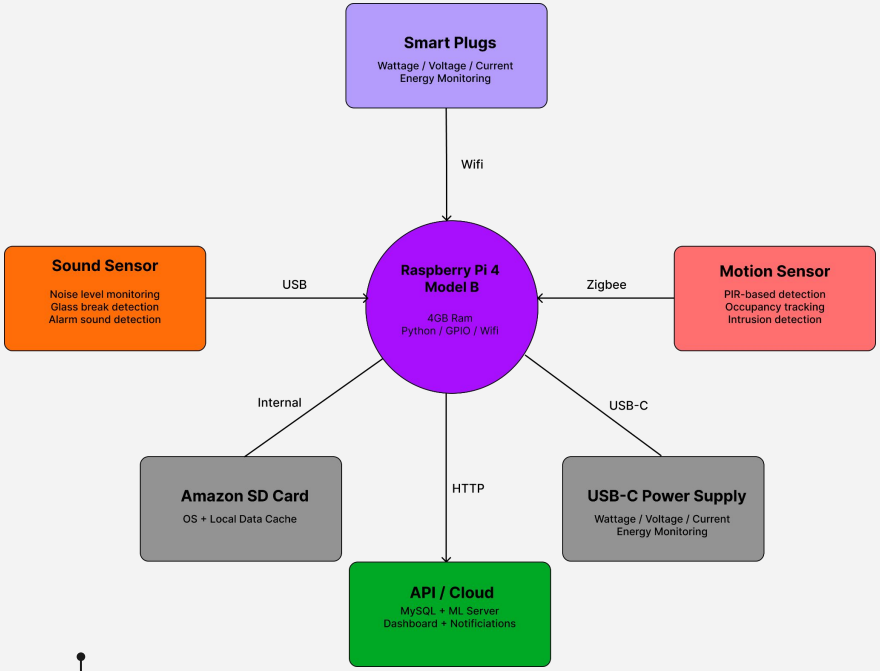
Data Pipeline

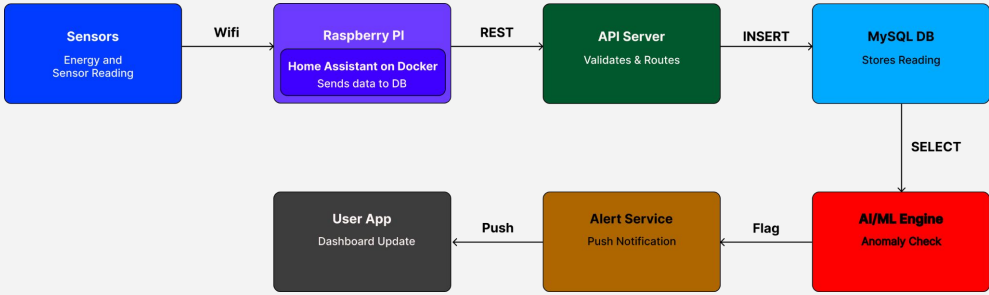
- All device data is routed through Home Assistant
- Forwarded to the AWS RDS database via the API

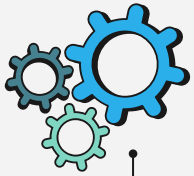
External Services

Emporia Application
TP - Link Kasa
Tailscale

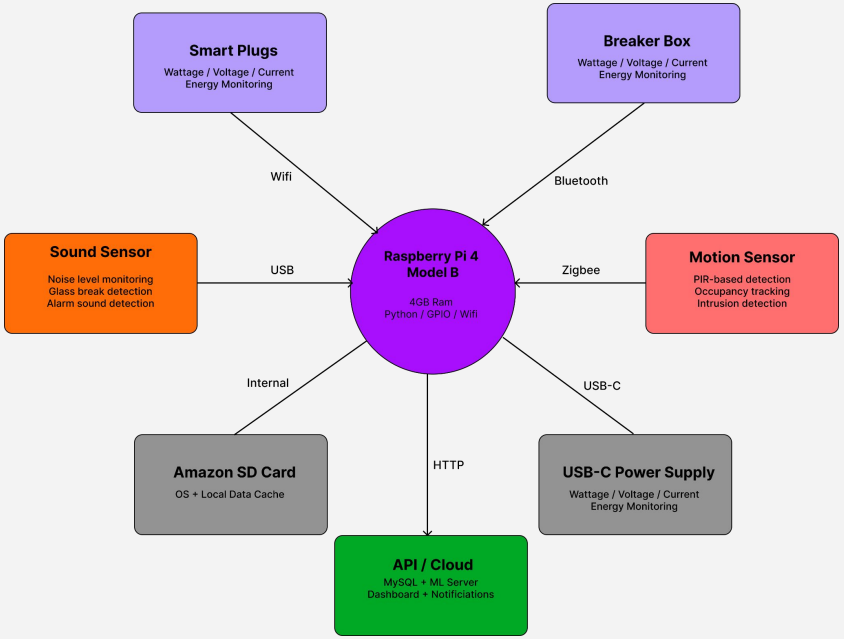
System-Level Description - Small House

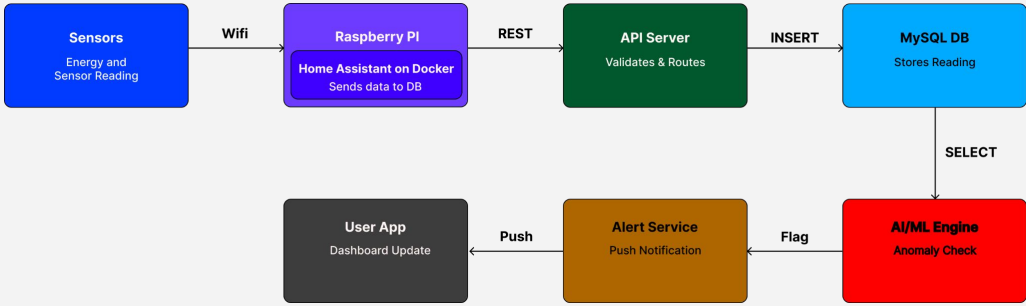


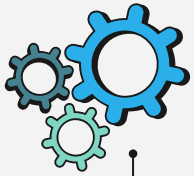




System-Level Description - Big House







Timeline

	Task Name	Dependencies	Start Date	Due Date
1	**Unit 1 and 2 Setup**	None	02/26/26	03/06/26
2	**Unit 3 Setup**	None	03/05/26	03/20/26
3	**Home Assistant Configuration**	Task 1, Task 2	03/01/26	03/05/26
4	**Database Setup**	Task 1, Task 2	02/28/26	03/05/26
5	**Data Collection**	Task 1, Task 2, Task 3	03/06/26	04/18/26
6	**API Setup**	Task 1, Task 2, Task 3	03/08/26	03/20/26
7	**Database Refinement**	Task 3, Task 4	03/18/26	03/28/26
8	**AI/ML Module**	Task 5, Task 7	03/18/26	04/05/26
9	**User Interface**	Task 4, Task 5	03/18/26	04/01/26
10	**Testing/Debugging**	Task 1-9	04/05/26	04/20/26

Vision For Product Demonstration

Our goal was to clearly demonstrate the connection between the smart home system and the practical benefits it provides to the user. The demonstration was designed around realistic scenarios that show how the system can assist with energy savings, home monitoring, and user convenience. By walking through these scenarios, the audience can easily understand how the system observes user behavior and provides helpful recommendations or alerts.

Vision For Product Demonstration

Examples:

Energy saving: We will train the model to know information such as cheapest electricity times, allowing you to "*set*" your dryer and have it run when electricity is the cheapest overnight.

Security: We intend to implement a "*vacation mode*". When the user indicates they are out of the house for a prolonged period of time, the smart home can process data differently and alert the user of any unexpected movement or sound.

Convenience: The system will be able to detect movement in a room without lights on when it knows the user is home, and based on this data can turn on the lights or *any* devices connected to the smart home.

Team Management

Frontend Software Engineer (William) has set up the Github integration with Tailscale. William has also setup the Home Assistant to run on a docker for the Raspberry PI in Unit 1. In addition, to also starting on the development of the frontend.

Machine Learning Engineer (Sriharsha) has worked on setting up AWS as well as documenting the AI models that will be used to implement the reasoning. Sriharsha has also worked tailoring data collection length to ensure that data retention is tailored towards the reasoning aspect of the project.

Lead Systems Engineer (Shiv) has worked on ensuring progress is being made and the timeline is updated respectively. Shiv has also ensured communication is consistent to all members, and also added home assistant onto a docker for Unit 2.

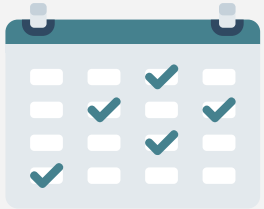
Hardware and Embedded Systems Engineer (Tyler) worked on setting up all the sensors and power devices in Unit 1 and the connection between the Kasa smart plugs and the phone application. Tyler also added home assistant onto a docker for Unit 3 and ensured unit 1 data was being collected.

Hardware/Machine Learning Engineer (Sutton) worked on setting up the sensors and power devices for Unit 2 and the connection between the kasa smart plugs and the phone application. Sutton also ensured that the data from unit 2 was being uploaded to the database.

Database and Backend Engineer (Evan) configured and deployed the SQL database on AWS for secure cloud storage and access. Evan also developed API endpoints for the Raspberry Pi to be able to transmit sensor data form unit 1 and 2 to the database.

Preliminary Results

id	device_id	home_id	timestamp	data
146357	5	1	2026-03-15 21:34:03	{"power": 7.542, "current": 0.152, "voltage": 122.187, "total_kwh": 0.459}
146356	4	1	2026-03-15 21:34:02	{"power": 6.198, "current": 0.124, "voltage": 120.86, "total_kwh": 0.093}
146355	3	1	2026-03-15 21:34:02	{"power": 0.629, "current": 0.02, "voltage": 121.998, "total_kwh": 0.01}
146354	2	1	2026-03-15 21:34:01	{"power": 0.686, "current": 0.014, "voltage": 121.938, "total_kwh": 0.011}
146353	1	1	2026-03-15 21:34:01	{"power": 0.0, "current": 0.0, "voltage": 122.083, "total_kwh": 0.0}
146352	5	1	2026-03-15 21:33:30	{"power": 7.539, "current": 0.15, "voltage": 122.239, "total_kwh": 0.459}
146351	4	1	2026-03-15 21:33:30	{"power": 5.425, "current": 0.107, "voltage": 120.531, "total_kwh": 0.093}
146350	3	1	2026-03-15 21:33:29	{"power": 0.661, "current": 0.02, "voltage": 122.08, "total_kwh": 0.01}
146349	2	1	2026-03-15 21:33:29	{"power": 0.652, "current": 0.014, "voltage": 121.915, "total_kwh": 0.011}
146348	1	1	2026-03-15 21:33:28	{"power": 0.0, "current": 0.0, "voltage": 122.046, "total_kwh": 0.0}



Thanks!

Do you have any questions?

This presentation utilized slidesgo